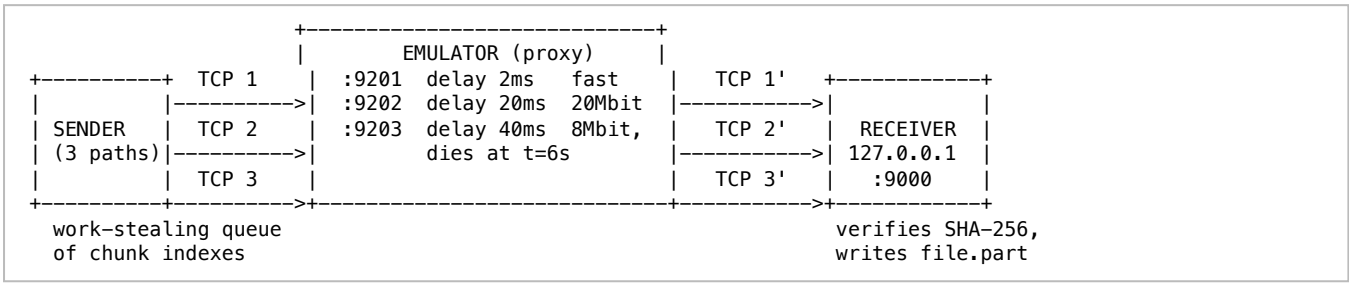
# Topology and Network Emulation

Real multipath networks are not available on one laptop, so each "path" is a TCP proxy (`mpft/emulator.py`) that sits between the sender and the receiver and degrades the traffic.




Each sender path is an independent TCP connection (own 3-way handshake, own congestion window). The proxy opens a second TCP connection to the receiver per accepted connection.

## Impairments (per path, JSON config)

| Field | Meaning |
|---|---|
| `listen_port` | Port the sender connects to for this path |
| `to` | `host:port` of the real receiver |
| `delay_ms` | One-way delay added in each direction (RTT grows by 2x this) |
| `jitter_ms` | Delay varies uniformly by +/- this; byte order is never changed |
| `rate_mbit` | Rate limit per direction, 0 = unlimited |
| `drop_after_s` | Path dies this many seconds after its first connection: listener and sockets are closed, new connections are refused. 0 = never |
| `corrupt_rate` | Probability per forwarded 16 KiB block (sender to receiver) that one bit is flipped |

## Run it

```
python -m mpft receive --listen 127.0.0.1:9000 --out downloads
python -m mpft emulate --config topology.example.json
python -m mpft send bigfile.bin --to 127.0.0.1:9201,127.0.0.1:9202,127.0.0.1:9203
```

## What is and is not emulated

- Emulated: latency, jitter, bandwidth limits, path failure, payload bit corruption.
- Not emulated: true IP packet loss. TCP hides loss by retransmitting, so on a real network loss shows up as lower throughput and higher RTT, which delay/rate limits approximate. Bit corruption is injected above TCP (TCP's checksum would normally catch it), to exercise MPFT's own SHA-256 and NACK path.